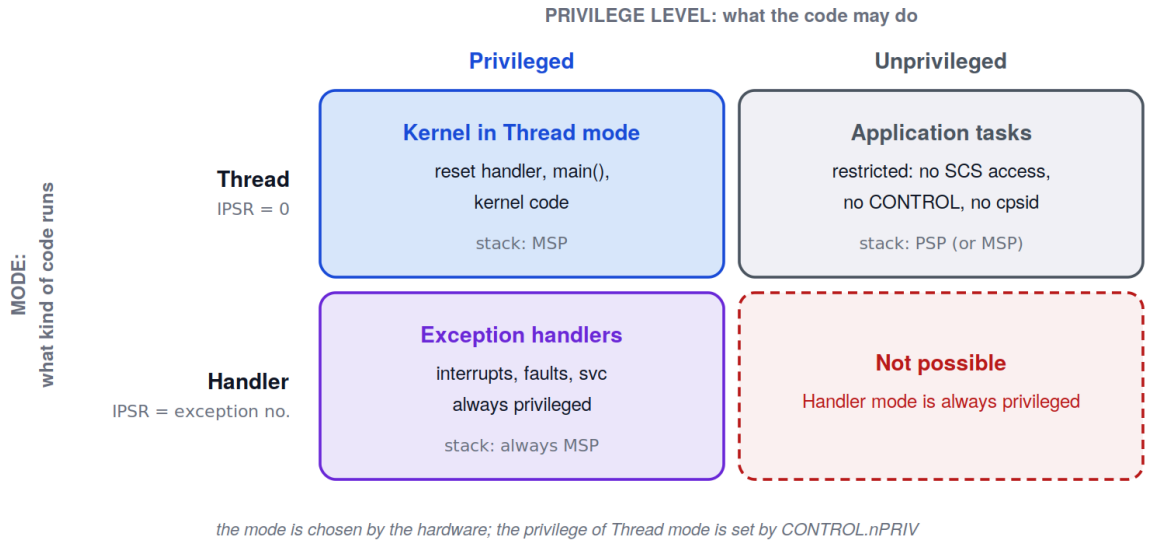
1 Execution Modes, Privilege, and Stacks

The Cortex-M3 implements in hardware the **separation** between **kernel mode** and **user mode** found in operating systems. Two independent properties describe the current state:

- the **mode** (what kind of code runs)
- the **privilege level** (what that code may do).

1.1 Thread Mode and Handler Mode



Mode and privilege: *two independent properties of the current state. Three of the four combinations exist; Handler mode is always privileged.*

1. **Thread mode** runs ordinary code: `Reset_Handler`, `main()`, and the functions they call; `IPSR` is 0.
2. **Handler mode** runs exception handlers; `IPSR` holds the exception number. The CPU enters Handler mode automatically when it takes an exception and returns to the previous mode when the handler finishes.

Mode	Privilege	Typical code
Thread	Privileged	`Reset_Handler`, kernel code after reset
Thread	Unprivileged	Application tasks after the kernel drops privilege
Handler	Always privileged	Every exception handler

> ONLY THREE COMBINATIONS PRIVILEGE CAN BE LOWERED ONLY IN THREAD MODE. HANDLER MODE IS ALWAYS PRIVILEGED, SO AN EXCEPTION ALWAYS RUNS PRIVILEGED AND RETURNS TO THREAD MODE WITH THE PRIVILEGE LEVEL IT HAD BEFORE.

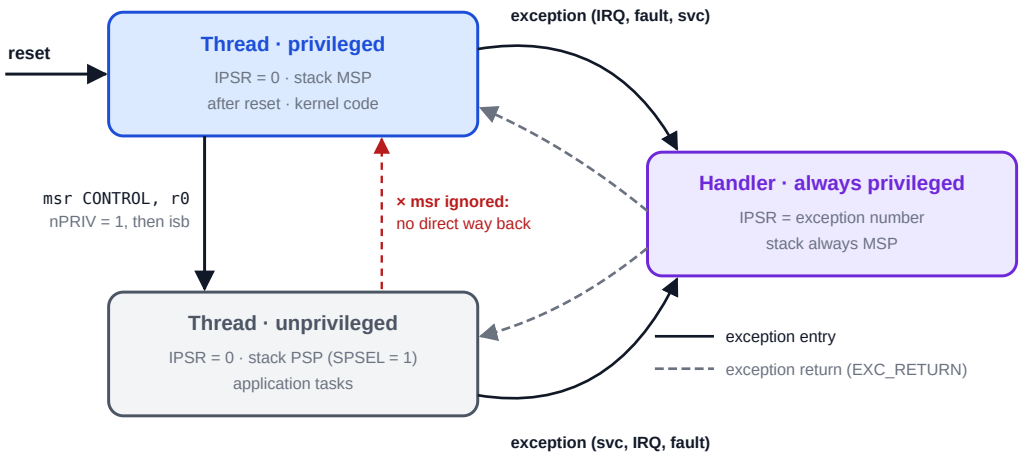


Figure 1.4: *Transitions between privileged Thread mode, unprivileged Thread mode, and Handler mode.*

After reset the CPU is in privileged Thread mode, which is why the boot code can configure the vector table, fault handling, and priorities.

1.2 Dropping Privilege

Privilege is dropped by setting `CONTROL.nPRIV` with the read-modify-write sequence shown in the CONTROL register section of the programmer's model chapter, followed by `isb`. The change is one-way: an unprivileged `msr control, r0` that tries to clear `nPRIV` is silently ignored, and the only ways back to privileged execution are a reset or an exception, whose handler runs in Handler mode.

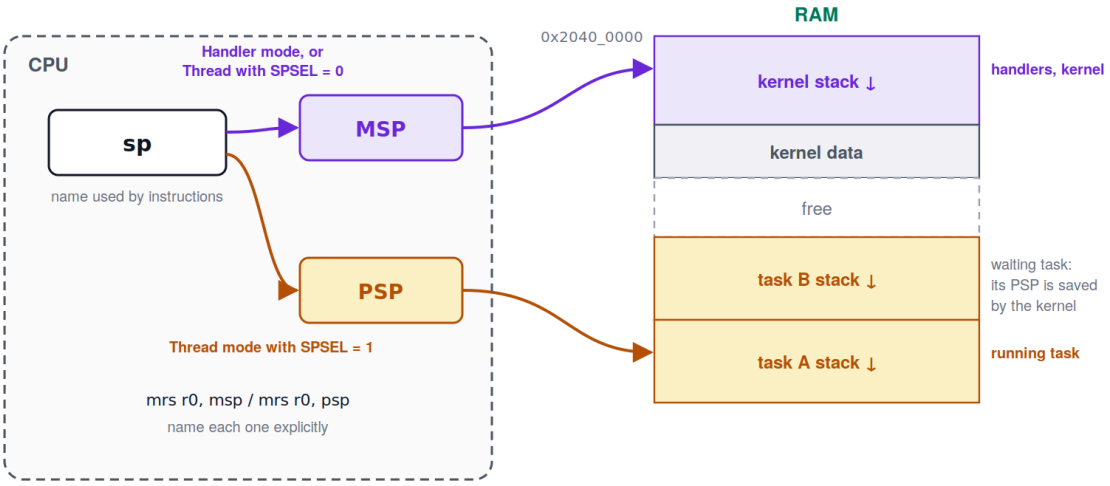
Unprivileged code keeps access to ordinary RAM, Flash, and its own code, but loses the ability to reconfigure or silence the machine:

Attempt by unprivileged code	Result
Access to the SCS (SysTick, NVIC, SCB, MPU)	BusFault
`msr control, r0` to regain privilege	Ignored
`cpsid i` / `cpsie i`	Ignored
Access to a region denied by the MPU	MemManage fault
Semihosting `bkpt 0xab`	Refused by QEMU by default: the fault escalates to HardFault

1.3 Main and Process Stack Pointers

The Cortex-M3 has two physical stack pointers:

1. The **main stack pointer** (`MSP`) is used after reset and always in **Handler mode**; it is the kernel's stack.
2. The **process stack pointer** (`PSP`) is used in **Thread mode** when `CONTROL.SPSEL` is 1; an operating system gives each task its own `PSP` stack. A task therefore typically runs with `CONTROL = 3` : unprivileged and on `PSP` .



an exception always switches to MSP: handlers and system calls never run on a task's stack, so a task that overflows its own stack cannot damage the kernel stack

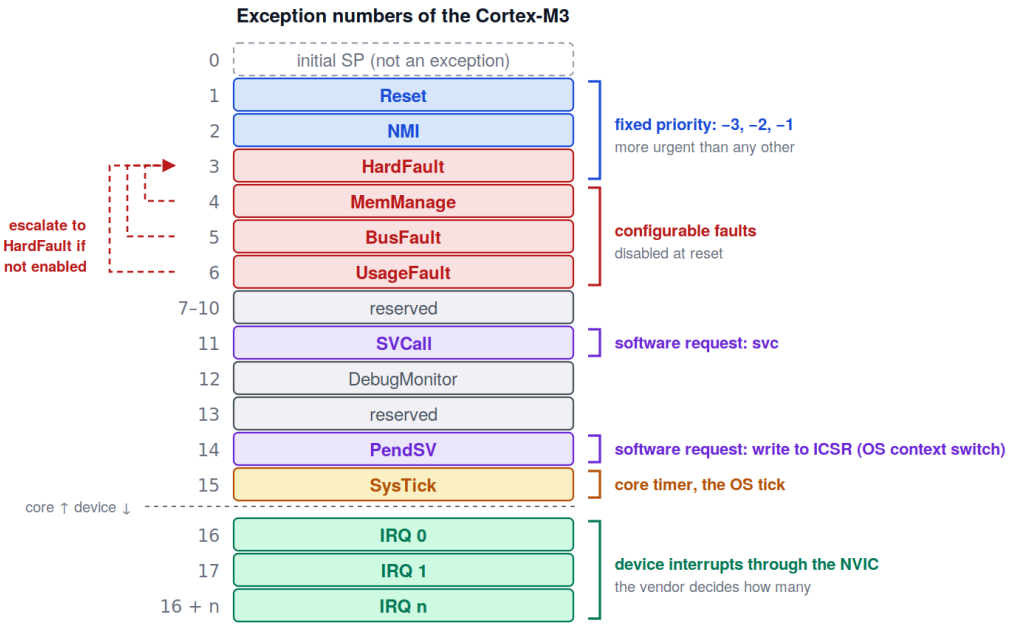
Main and process stack pointers: *sp refers to MSP or PSP depending on the mode and on CONTROL.SPSEL. MSP points to the kernel stack, PSP to the stack of the running task.*

The register name `sp` always refers to the stack pointer currently selected, while `mrs r0, msp` and `mrs r0, psp` name each one explicitly. Because Handler mode always switches to `MSP`, interrupt handlers and system calls never execute on a task's stack: a task that corrupts or overflows its own stack cannot corrupt the stack the kernel runs on.

2 Exception Model

An **exception** is any event that makes the **CPU suspend** the current code and **run a specific handler** in **Handler mode**. An **interrupt** is an **exception** raised by a **hardware device**. Faults, system calls, the system timer, and device interrupts all go through the same mechanism.

2.1 Exception Types and Numbers



Exception numbers: *numbers 1–15 belong to the core and are the same on every Cortex-M3; device interrupts start at 16. Disabled configurable faults escalate to HardFault.*

Number	Exception	Raised by
1	Reset	Power-on, reset pin
2	NMI	Non-maskable interrupt (e.g. watchdog expiry)
3	HardFault	Any fault with nowhere else to go
4	MemManage	Violation of MPU rules
5	BusFault	Access to an address that does not answer or refuses the access
6	UsageFault	Undefined opcode, Thumb bit 0, division by zero (if trapped)
7–10, 13	Reserved	—
11	SVCall	The `svc` instruction
12	DebugMonitor	Debug events in monitor mode
14	PendSV	A software request (`ICSR.PENDSVSET`)
15	SysTick	The core timer
16 + n	IRQ n	Device interrupt line n, through the NVIC

Numbers 0–15 belong to the core and are identical on every Cortex-M3; device interrupts follow. Number 0 is not an exception: its vector table slot holds the initial stack pointer.

At reset only HardFault is active among the faults: MemManage, BusFault, and UsageFault are disabled, so every fault *escalates* to HardFault until the boot code enables the dedicated handlers.

PendSV deserves a note because of its role in operating systems. It is raised only by software, by writing a bit in a

system register, and it is normally given the lowest priority. An OS uses it to request work, typically a context switch, that must run only after every more urgent handler has finished.

2.2 Priorities, Preemption, and Lockup

When several exceptions compete, the more urgent one runs first. On Cortex-M a **lower number means more urgent**. A pending exception preempts the running code only if its priority is *strictly* more urgent; otherwise it stays pending until the current handler returns. Preemption of one handler by another is called *nesting*, which gives the **NVIC (Nested Vectored Interrupt Controller)** its name.

Implementations do not use all eight bits of a priority byte. Real chips implement only the upper 3 or 4 bits, and the lower bits read as 0, so distinct priorities must differ in their top bits (`0x00` , `0x20` , ..., `0xE0` for 3 bits).

> ⚠ **LOCKUP**
>
> If a fault occurs while the CPU already runs at HardFault priority, or the fault cannot be reported because exception entry itself fails (for example, the stack pointer points outside RAM), there is nowhere to escalate. The CPU enters *lockup* and stops executing. QEMU reports it as `Lockup: can't escalate 3 to HardFault (current priority -1)` .

2.3 Vector Table and Reset Sequence

The vector table is an array of 32-bit words where slot N holds the address of the handler of exception N . When exception N occurs, the CPU reads the slot and branches there. Slot 0 is special: it holds the initial value of `MSP` , not a code address. The table is located through the `VTOR` register, which resets to `0x0000_0000` ; at reset it must therefore be at the start of the Code region, which is why the linker places it first in Flash.

When the Cortex-M3 leaves reset, before any software runs, the hardware performs exactly two reads:

1. the word at address `0x0` is loaded into `MSP` ;
2. the word at address `0x4` is loaded into `PC` , and execution starts there in privileged Thread mode.

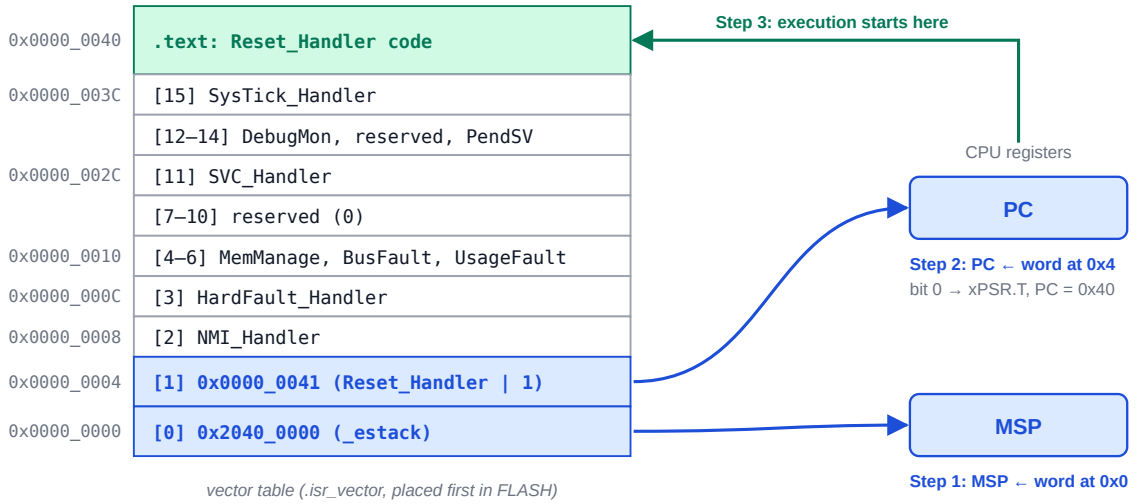


Figure 1.5: *Reset sequence: the CPU loads MSP from address 0x0 and PC from address 0x4, then executes Reset_Handler.*

There is no BIOS, no loader, and no C runtime. Initializing memory, configuring the core, and calling `main()` are the job of the code whose address is stored in slot 1, `Reset_Handler`.

Offset	Slot	Content
`0x00`	0	Initial `MSP` (e.g. `0x2040_0000`, one byte past the end of RAM)
`0x04`	1	`Reset_Handler` address, with bit 0 set
`0x08`	2	`NMI_Handler`
`0x0C` – `0x18`	3–6	HardFault, MemManage, BusFault, UsageFault handlers
`0x2C`	11	`SVC_Handler`
`0x38`, `0x3C`	14, 15	`PendSV_Handler`, `SysTick_Handler`
`0x40` + 4n	16 + n	Handler of IRQ n

Every handler address is stored with the Thumb bit set. Reserved slots hold 0, and every other slot must point to valid code even if the exception is never expected; startup code usually fills unused slots with a default handler that loops forever, so that an unexpected exception stops at a recognizable address.

> **FROM IRQ NUMBER TO VECTOR SLOT IRQ N IS EXCEPTION 16 + N, AND ITS HANDLER ADDRESS IS AT OFFSET 4 × (16 + N). IRQ 1 IS EXCEPTION 17, AT OFFSET `0x44`.**

2.4 Exception Entry and the Stacked Frame

Before the first instruction of a handler, the hardware:

1. pushes 8 words, the *stacked frame*, onto the stack in use when the exception occurred (`MSP` or `PSP`);
2. writes a special `EXC_RETURN` value into `lr`;
3. reads the handler address from the vector table and branches to it in Handler mode, on `MSP`.

Offset from new `sp`	Stacked register
`+0x00`	`r0`
`+0x04`	`r1`
`+0x08`	`r2`
`+0x0C`	`r3`
`+0x10`	`r12`
`+0x14`	`lr` of the interrupted code
`+0x18`	`PC` : where the interrupted code resumes
`+0x1C`	`xPSR`

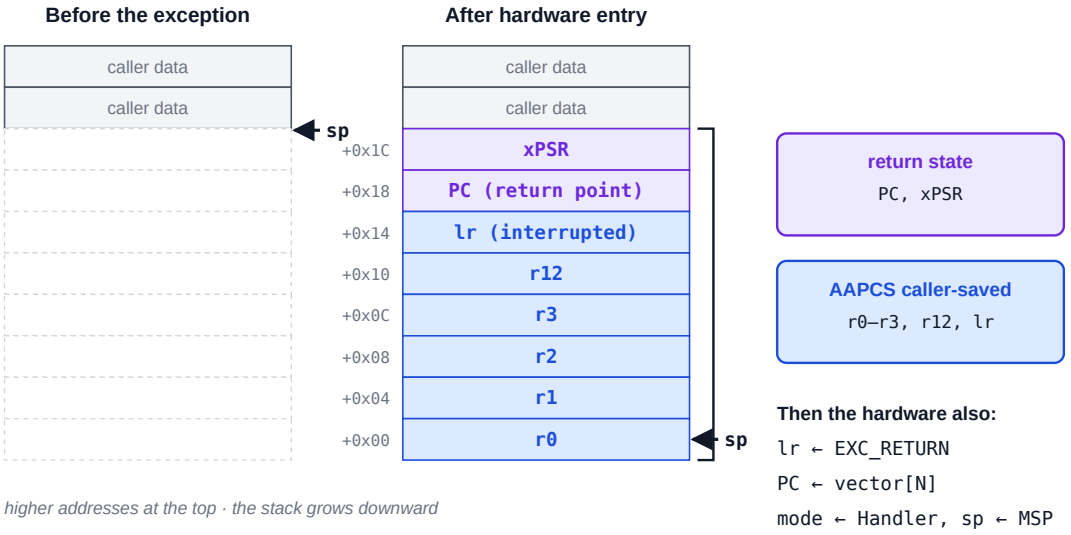


Figure 1.6: *Stack before and after exception entry, with the 8-word stacked frame pushed by hardware.*

The hardware saves exactly the AAPCS caller-saved registers plus `PC` and `xPSR`, and any compiled C function already preserves the callee-saved `r4` – `r11`. Together, these two facts mean that an exception handler can be an ordinary C function, with no special prologue or epilogue. When `CCR.STKALIGN` is set (its reset value on the Cortex-M3), the hardware also aligns `sp` to 8 bytes on entry, as the AAPCS requires, and records the adjustment in bit 9 of the stacked `xPSR`. Entry takes about 12 cycles.

2.5 Exception Return through EXC_RETURN

A handler ends with an ordinary return instruction (`bx lr` or `pop {..., pc}`), yet the CPU must restore the saved state rather than jump to a return address. While a handler runs, `lr` holds a value of the form `0xFFFFFFxx`, where no code can exist. Loading such a value into `PC` is recognized as an *exception return*: the CPU pops the 8-word frame from the indicated stack, restores the mode, and resumes at the stacked `PC`.

EXC_RETURN	Return to	Frame popped from
0xFFFFFFF1	Handler mode (a preempted handler)	MSP
0xFFFFFFF9	Thread mode	MSP
0xFFFFFFFD	Thread mode	PSP

Bits 31–4 are all ones and mark the value as `EXC_RETURN`. Bit 3 selects the return mode (1 = Thread, 0 = Handler) and bit 2 the stack (1 = `PSP`, 0 = `MSP`). On entry, bit 2 therefore tells the handler which stack holds the frame just pushed, which a system-call handler needs in order to read the caller's arguments.

2.6 Tail-Chaining and Late Arrival

When exceptions occur close together, the core avoids saving and restoring the same state repeatedly.

Optimization	Situation	Behaviour
Tail-chaining	A handler ends while another exception is pending	No pop and re-push of the frame: direct jump to the next handler (about 6 cycles instead of about 24)
Late arrival	A more urgent exception arrives while the frame of a less urgent one is being pushed	The frame is reused; the more urgent handler runs first, then tail-chains to the other

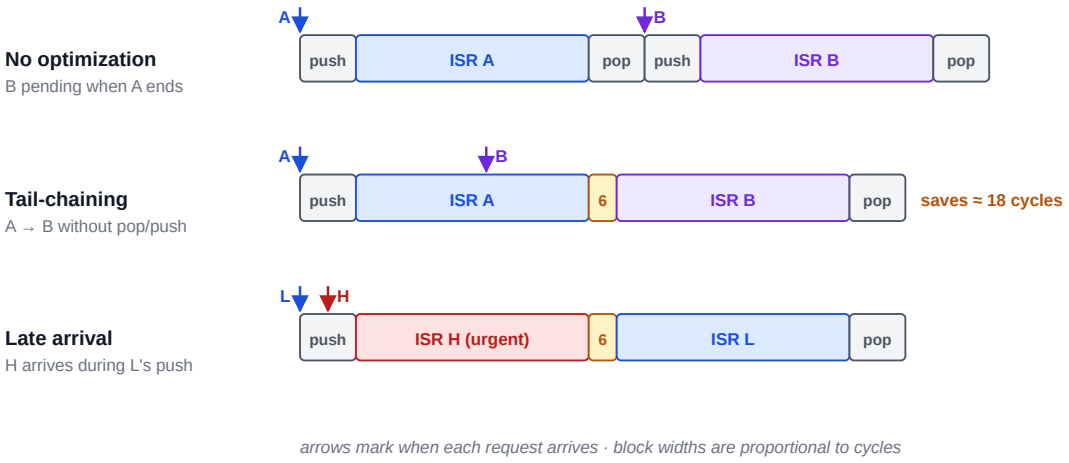


Figure 1.7: *Timelines of two exceptions without optimization, with tail-chaining, and with late arrival.*

2.7 SVC: A Controlled Entry into Privileged Code

```
void SVC_Handler( void )
{
    /* privileged service code */
}

/* in unprivileged Thread mode */
__asm__ volatile ( "svc #0" : : : "memory" );
```

`svc #n` (supervisor call) is one of the few things unprivileged code can always do. It raises SVCall (exception 11) synchronously, so the handler in slot 11 runs in Handler mode and therefore privileged. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and still unprivileged. Inside the handler `CONTROL.nPRIV` still reads 1: the handler is privileged because of the mode, not because the bit changed.

Privilege is borrowed only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction and, together with arguments in `r0` – `r3`, can select the requested service. This is the hardware foundation of every system call: restricted code cannot touch the machine directly and must go through a single, controlled entry point.

> **MISSING HANDLER**
>
> If `SVC_Handler` is not defined, `svc` reaches the default handler and the program hangs silently. Unlike a fault handler, `SVC_Handler` returns normally, because `svc` is a deliberate request, not an error.